



Defining Test Data Constructs and Exploring LLMs for Data Generation

T1000

For the Semester Project for the

Bachelor of Science

of the degree program New Study Informatik

at the Baden-Württemberg Cooperative State University

from

Liam Thomson

September 2024

Processing period	26 Weeks
Matrikulation number, Course	3248571, DHBW-TINFO23
Training Company	Accenture, Kronberg im Taunus
Supervisor	Piotr Preuss
Examiner	Dr. Prof. Kay Berkling

Confidentiality Notice

The present T1000 thesis titled *Generating Test Data Construct and Looking for Opportunities to use LLMs for Data Generation* is classified as confidential and is submitted solely for examination purposes in the New Study Informatik program at the Baden-Württemberg Cooperative State University at the submission location. Any inspection or publication – including partial excerpts of the thesis – requires prior approval from Accenture.

Declaration

I hereby declare under oath:

1. that I have completed my T1000 mit dem Thema on the topic *Generating Test Data Construct and Looking for Opportunities to use LLMs for Data Generation* without external assistance;
2. that I have indicated the adoption of literal quotations from the literature and the use of the ideas of other authors at the corresponding points within the work;
3. that I have not submitted my T1000 for any other examination;

I am aware that a false declaration will have legal consequences.

Baden-Württemberg Cooperative State University, September 2024

Liam Thomson

Abstract

This project demonstrate a Python-based solution to define and structure Test Data Structures for testing a Billable Process in SAP S/4HANA. The primary goal was to automate the generation of critical fields such as customer details, addresses, and energy market data, categorizing these fields as required, optional, or auto-generated. Simple iteration techniques were applied to fields like last names, while more advanced approaches, such as online generators and Large Language Models, were explored for complex data types like names, addresses, and IBANs.

Table of Contents

1 Introduction **1**

2 Literature Review **3**

3 Methodology **4**
- 3.1 SAP System Overview 4
- 3.2 Defining Data Constructs 9
- 3.3 Python Script 10

4 Implementation **12**
- 4.1 Creating a Bilp in the System 12
- 4.2 Extracting the Tables 12
- 4.3 Categorising and Trimming the Tables 15
- 4.4 Writing the Python Script 16
- 4.5 Data Generation 17

5 Discussion **20**

6 Conclusion **22**

Abbildungsverzeichnis **i**

List of Tables **ii**

Listings **iii**

References **iv**

Abkürzungsverzeichnis **vi**

1 Introduction

The utilities sector in Germany is undergoing significant transformation, driven by the increasing adoption of renewable energy sources, and the need for enhanced efficiency in resource management. To navigate this dynamic environment, companies in the utilities sector require robust digital solutions that can adapt to evolving market demands. SAP S/4HANA, an ERP[1] suite built on the in-memory SAP HANA platform, has emerged as a leading solution for enterprises seeking to streamline operations, enhance data analytics, and integrate processes across the value chain.

A SAP test environment was made available for TAP[2], the project I am staffed on in Accenture. (AG, n.d.) "TAP is a billing platform developed by Thüga, a leading municipal utility company in Germany, in collaboration with Accenture. TAP is specifically designed for the energy sector to streamline and optimize the billing processes for energy providers, especially municipal utilities."

[1] Enterprise Resource Planning
[2] Thüga-Abrechnungsplattform

The primary objective of this project is to develop a Python-based solution to define and structure the TDC[1] used for testing BilP[2]s in SAP and seeing if an LLM[3] could potentially be used for data generation. The project aims to categorize and highlight critical fields such as customer information (name, address, bank details) and energy market data, determining which fields are strictly required, optional, or auto-generated. These fields must be clearly marked as either incrementally generated, unique, or potentially suitable for automated value generation through AI. The ultimate goal is to create a program that automates the generation of these values, ensuring that each TDC is unique and properly configured for future testing scenarios. However, the actual execution of the testing is beyond the scope of this project.

The challenge in this project arises from the need to accurately identify and categorize the required fields for a BilP. Some fields are automatically generated and carried over between process steps, while others require manual input. Developing a Python-based prototype to manage this data construct programmatically adds a layer of complexity, as the program must distinguish between required, optional, and auto-generated fields. Additionally, for cases where field values cannot be incrementally generated, integrating AI to automate the generation of unique values becomes essential. This project focuses on building the data structure and automating the value generation process, ensuring that each TDC is distinct and fit for testing.

1 Test Data Construct
2 Billable Process
3 Large Language Model

2 Literature Review

Enterprise Resource Planning (ERP) systems integrate various business processes, providing a unified platform to enhance operational efficiency (Molina-Castillo, Rodríguez, López-Nicolas and Bouwman, 2022). Recent advancements in Large Language Models (LLMs) offer promising applications for test case construction, particularly in the automated generation of test data, enabling large-scale testing with minimal human intervention (de Lima Junior rflj@cesar.org.br Luiz Fernando Paes de Barros Presta lfpbp@cesar.school Lucca Santos Borborema lsb@cesar.school Vanderson Nogueira da Silva vns@cesar.org.br Marcio Leal de Melo Dahia mlmd@cesar.org.br Anderson Carlos Sousa e Santos acss@cesar.org.br, 2023). OpenAI's research demonstrates the evolving capabilities of LLMs, further emphasizing their potential to transform test data generation in ERP environments (OpenAI, 2024).

The use of LLMs for synthetic tabular data generation faces limitations in capturing functional dependencies between columns, a critical aspect for accurate data representation (Anonymous, 2024).

3 Methodology

3.1 SAP System Overview

The test system my project is based on SAP S/4HANA (TechTarget, 2023) SSAP S/4HANA is an ERP business suite based on the SAP HANA in-memory database that allows companies to perform transactions and analyse business data in real time. "

I'm accessing the SAP system through the SAP GUI 8.00. It may come as a surprise, but getting to grips with the SAP system and defining the BilP was the most challenging part of this project, especially considering a project restructure was thrown in the mix.

The SAP system I'm working with is divided into two main parts, based on the type of customer it handles. These parts are referred to as **Client 300** and **Client 400**, and each plays a distinct role within the system.

3.1.1 Client 300

This part of the system deals with individual consumers, typically household customers using energy services like electricity, gas, or water for personal consumption. The interaction here is fairly straightforward, focusing on utility use, billing, and service management. The data stored in this section includes personal information such as name, address, and all other system information like contract account, contract, installation, device infoset, and device location. The BilP for Client 300 is pretty standard, with regular payments based on their consumption, which is monitored over time.

3.1.2 Client 400

The second part of the system, Client 400, is designed for businesses and network operators. These partners usually consume energy on a larger scale or are involved in distributing utilities across regions to 300 clients. The data for Client 400 is more complex because it includes things like contract management, metering points, and MaLo[1] / MeLo[2]. This allows the utility company to track, manage, and bill these large-scale consumers effectively.

3.1.3 Transaction Code Table

According to (Panaya, 2024), SSAP Tcode[3] are short alphanumeric strings that represent specific entry points within an SAP system. They allow users to navigate through different SAP modules and perform tasks such as creating sales orders, generating reports, or managing inventory. Each Tcode triggers a sequence of predefined steps and processes, enabling users to access and manipulate data stored in the SAP system effectively."

(Seren, 2024) "The following table shows the needed Tcode for the BilP in SAP. For all transactions there is also example data available, which is used for this documentation. The transactions are also ordered by the number of steps."

1 Market Location
2 Metering Location
3 Transaction Code
1 Sample data is based on test system and is Subject to change.

No.	Tcode	Name	Sample Data
1	BP	Business Partner Mgmt.	1300000819
2	CAA1	Create contract account	002310001112
3	ES55	Create connection object	7300000074
4	ES60	Create premise	6300000685
5	ES30	Create installation	4300001320
6	IQ01	Creating ISU Equipment	83426864608 (random)
7	EG44	Create device info set	158000000011000133
8	ES65	Create device location	8300000029
9	EEDM10	Metering Point (MaLo/MeLo)	DE827304759998273047599827304759
10	EG34	Build in (billing-wise)	See device in installation
11	EC50E	Create move-in (contract)	3310003034
12	EL01	Create reading order	See Installation, billing-view
13	EL28	Create reading	See Installation, billing-view
14	EASIBI	Create a bill	See Installation, billing-view

Table 3.1: Tcodes and Sample Data[1]

3.1.4 300 and 400 Transaction Steps and Definitions

These Tcode are shared between Client 300 and 400. Though note that in practice some of the data entered will vary between the two client types depending on the Tcode.

BP

The **BP** Tcode is used to manage and maintain Business Partners in the SAP system. A Business Partner is any entity that interacts with the utility company, whether they are individual consumers or business entities. This transaction primarily handles personal details such as the customer's name, address, communication preferences, and bank account information.

CAA1

CAA1 is used to create a **contract account**, which is linked to the Business Partner and handles the financial aspects of the relationship. For Client 300, this would involve setting up the payment terms for utilities consumed by the individual (such as electricity, gas, or water). The Contract Account is the foundation for billing cycles and payments, ensuring that the customer is billed appropriately based on their consumption.

ES55

The **connection object** is used to connect the BP to the Contract Account.

ES60

Creates the **premise** which represents the customers location where the energy is consumed. The data is established by linking the connection object.

ES30

The **installation** includes all necessary equipment, such as meters, and is linked to the premise and the BP. Each installation has its own unique number.

EG44

A **device infoset** refers to the technical data related to a metering device installed at a customer's location to measure utility consumption.

ES65

The **device location** is used to define the metering devices location.

EG34

Connects a **device to an installation**.

EC50E

Creates a **contract** which links all the above steps for Client 300.

A client may be billed at this point.

3.1.5 400 Transaction Steps and Definitions

From this point on the Tcode are exclusive to Client 400.

EEDM10

Creates and connects **metering points** to the installation for MaLo[1] and MeLo[2].

IQ01

Creates an **ISU-Equipment** which refers to the physical metering or measurement device installed at a customer's location.

EL01

Creates a **reading order** which plans a meter reading at a specific date and interval.

EL28

Creates the actual **meter reading**.

[1] Market Location
[2] Meassurement Location

EASIBI

Creates an **invoice**.

ES32

Gives an overview of **billing information.**

SE16N

Is used to check table's value and/or extract data.

3.2 Defining Data Constructs

In the SAP testing environment, I focused on the BilP already in place. The goal was to identify the required fields, categorize them, and set up a system to generate test data, ensuring each construct was unique.

3.2.1 Going Through the BilP Chronologically

To create test data in the system, I went through the BilP as defined by (Seren, 2024) step by step with test data, once for Client 300 and once for Client 400.

3.2.2 Categorizing the Data

It was a clear requirement from the start of the project that the data entered would need to be categorized. It wasn't exactly clear how or why until entering the data into the system was put into practice.

3.2.3 Extracting SAP Table Information

Using Tcode SE16N, I pulled the tables for each Tcode, starting with the BP table (BUT000). This gave me the actual column names and formats that SAP uses to store the data. Once I had that info for each Tcode, I could get a clearer picture of how the system was handling the data behind the scenes.

3.2.4 Trimming the Tables

After pulling the table data, I stripped out any optional columns that weren't necessary for the BilP. I was left with only the essential fields. I then organized these fields by their data types and properties—predefined values, formula-based fields, strings, and auto-generated data—so I could manage them better in the next steps.

3.3 Python Script

3.3.1 Conceptualisation

With the tables sorted and categorized, I started designing a Python script to model the data construct for each table. Each column became a key in a python nested dictionary (**?**) "Dictionaries are used to store data values in key:value pairs", and the corresponding value was either generated or predefined based on the column's type.

3.3.2 Data Generation

For string fields (like names), I could set up a pipeline to send prompts to an LLM[1] to generate realistic values, however, strictly speaking for this test system we don't actually need a realistic name for a successful BilP, so the easiest solution was to just incrementally update the last name. This subject is later discussed in chapter 6. For fields requiring a specific format, like IBAN, an open source generator could be implemented in various ways like (apilayer, n.d.) for IBAN and (Fakexy.com, 2024) for address. Auto-generated fields were left out as the test system generated those.

[1] Large Language Model

3.3.3 Outputting and Storing the Data Constructs

Once all the values for a table were generated, the script would store the entire construct in a nested Dictionary, mimicking the hierarchy of the data in the test system. This way, I could easily visualise and generate large amounts of unique test data, which could be used for testing the BilP end-to-end within SAP.

In the end, this approach allowed me to demonstrate a streamlined system to generate and manage test data for the BilP, ensuring everything fit SAP's requirements while still keeping the data unique and properly categorized.

3.3.4 Script Prototype

To demonstrate the approach, I wrote a small pseudo-code script focused on a subset of the data construct, specifically for the BP. The goal was to create a manageable way to represent the required fields, and handle their value generation efficiently.

4 Implementation

4.1 Creating a Bilp in the System

Entering BP in the search box in the top left of the screen would open the data entry fields for BP as the filled in depicted example in figure 4.1. Once values for personal data, address, and bank details where entered, the page could be saved and if successful a BP ID would be generated: 1400000535.

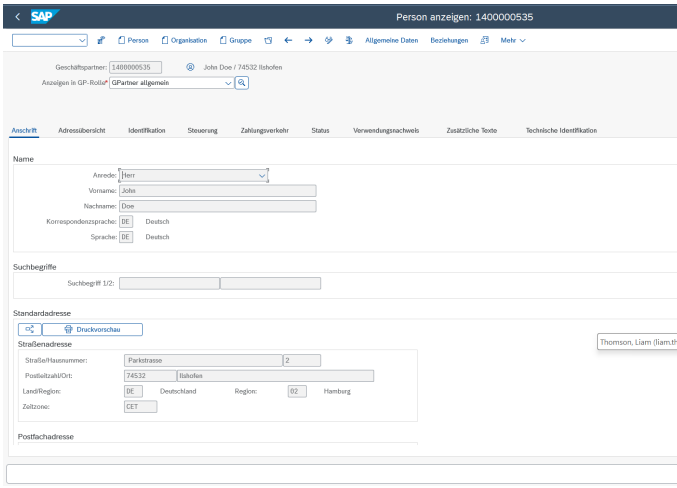


Figure 4.1: Creating a BP[1]

4.2 Extracting the Tables

Entering Tcode SN16N, then entering table name BUT000 (table name for BP), and then querying the BP ID "1400000535"we get the master table with the sample data. This we can export to excel with "Tabellenkalkulation/Spreadsheet calculation". These steps were repeated for all 14 Tables.

Table 4.1: Field Names and Values Table[1]

Field Name	**Value**
address.name.title	Mr.
address.name.firstName	John
address.name.lastName	Doe

Field Name	Value
address.name.correspondenceLang	DE
address.name.language	DE
address.searchTerms.searchTerm1	
address.searchTerms.searchTerm2	
address.standardAddress.streetAddress.streetHouseNumber	Parkstrasse 2
address.standardAddress.streetAddress.postalCode	74532
address.standardAddress.streetAddress.city	Ilshofen
address.standardAddress.streetAddress.countryRegion	DE
address.standardAddress.streetAddress.region	02 Hamburg
address.standardAddress.streetAddress.timeZone	CET
address.communication.telephone	
address.communication.telephone.extension	
address.communication.cellPhone	
address.communication.fax	
address.communication.email	
address.communication.comments	
address.communication.addressValidFrom	05.09.2024
address.communication.addressValidTo	31.12.9999
paymentTransactions.bankDetails.IBAN	DE30 5001 0517 6357 5412 57
paymentTransactions.bankDetails.bankCountryRegion	DE
paymentTransactions.bankDetails.bankKey	50010517
paymentTransactions.bankDetails.bankNumber	50010517

Field Name	Value
paymentTransactions.bankDetails.swiftBIC	INGDDEFFXXX
paymentTransactions.bankDetails.bankAccount	6357541257
control.notes.Correspondence	
control.notes.AccountingNote	
control.notes.MarketingNote	
control.notes.BusinessPartnerInfo	
control.notes.BusinessHours	
control.notes.NoteLog	
control.notes.IPMTest	
control.notes.CreditManagement	
control.parameters.BPType	
status.archivingFlags.ArchivingFlag	
status.lock.CentralBlock	
identification.personalData.Sex	

4.3 Categorising and Trimming the Tables

As I entered the test data into the system, it became clear how the fields could be grouped:

- **Predefined Values**: Drop-downs with specific options like title and language.
- **Formula-Based Values**: Fields like IBAN or email addresses that had strict validation requirements.
- **String Values**: General text fields, such as names, addresses, and descriptions.
- **Auto-Generated Values**: Fields the system handles automatically, like unique IDs or timestamps.
- **Optional Values**: Fields that can be left blank without causing errors.

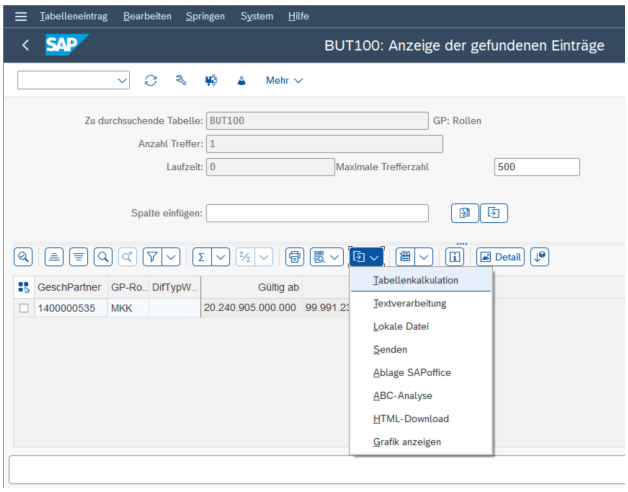


Figure 4.2: Extracting BP Master Data[1]

Taking one look at the extracted BP table, we'll find a lot of empty values. these fields can safely be removed as they are not necessary to complete the BilP.

After this process for BP we are left with the following set in table 4.2.

Field	Description
title	Predefined values; Mr, Ms
firstName	Static string: TestUser
lastName	Variable String: 00000001
correspondenceLang	Predefined values; DE German, EN English, ...
language	Predefined values; DE German, EN English, ...
streetHouseNumber	Variable String imported from address generator
postalCode	Variable String imported from address generator
city	Variable String imported from address generator
countryRegion	Variable String imported from address generator
region	Variable String imported from address generator
iban	Variable String imported from IBAN generator

Table 4.2: BP Fields and Data Categories

4.4 Writing the Python Script

4.4.1 Data Structure

Setting up the nested dictionary was straight forward enough. I simply mimicked the Hierarchical structure of the test system.

```
bp_mapped = {
    'address': {
        'name': {
            'title': 'Mr.',
            'firstName': 'John',
            'lastName': 'Doe',
            'correspondenceLang': 'DE',
            'language': 'DE'
        },
        'standardAddress': {
            'streetAddress': {
                'streetHouseNumber': 'Parkstrasse 2',
```

```
                'postalCode': '74532',
                'city': 'Ilshofen',
                'countryRegion': 'DE',
                'region': '02 Hamburg'
            }
        }
    },
    'paymentTransactions': {
        'bankDetails': {
            'iban': 'DE30 5001 0517 6357 5412 57'
        }
    }
}
```

Listing 4.1: Mapped Python Dictionary for Business Partner

4.5 Data Generation

Last Name Iterator

This is how simple it could be to iterate and return an 8 digit number.

```
def last_name_iterator(start=1, end=100, num_digits=8):
    for i in range(start, end + 1):
        yield str(i).zfill(num_digits)
```

Listing 4.2: Last Name Iterator Function

4.5.1 Address Generator

The address details where simply copy and pasted from (Fakexy.com, 2024) for demonstration purposes. See figure 4.3.

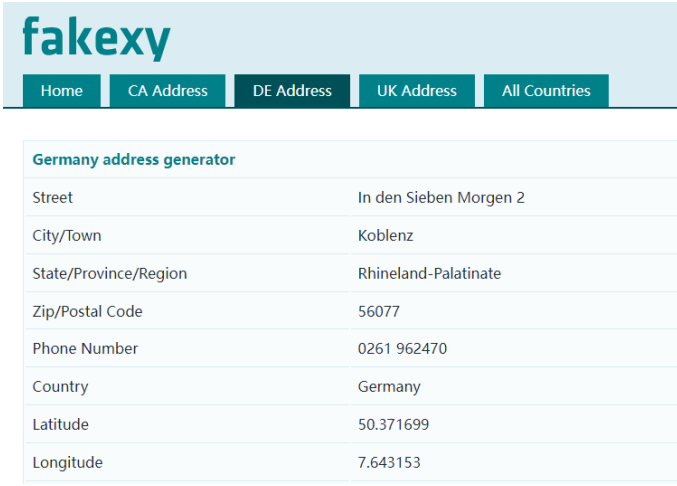


Figure 4.3: Extracting Address Data[1]

4.5.2 Iban Generator

As specific logic lies behind the IBAN generation which is checked in the test environment, simply iterating or relying on LLM wont suffice. The free online generator (apilayer, n.d.) was used for this purpose.

4.5.3 API

A potential solution for retrieving generated address and bank data is to use an API from sources like (RapidAPI, 2024), where the curl[2] would look like listing 4.3, and the response would be in JSON[3] format.

```
curl --request GET \
  --url 'https://dawn2k-random-german-profiles-and-names-
      generator-v1.p.rapidapi.com/?gender=b&cc=all&email=gmail.
      com%2Cyahoo.com&pwlen=12&ip=a&phone=l%2Ct%2Co&seed=
      helloworld&format=json&count=10&maxage=40&minage=30&uuid=
      true&color=true&lic=true&images=true' \
  --header 'x-rapidapi-host: dawn2k-random-german-profiles-and
      -names-generator-v1.p.rapidapi.com' \
  --header 'x-rapidapi-key: Sign Up for Key'
```

Listing 4.3: Curl Request for Random German Profiles

[2] Client URL
[3] JavaScript Object Notation

4.5.4 LLM Generation

Here is a Python pseudo-code for sending prompts to an LLM and storing the responses in a dictionary:

```
def send_prompt_to_llm(prompt):
    # Simulate sending the prompt to the LLM and getting the
        response
    # In actual implementation, this would be an API call to
        the LLM
    response = llm_api_call(prompt)
    return response

prompts = [
    "Generate a first name",
    "Generate a last name",
    "Generate an IBAN",
]

for prompt in prompts:
    # Send the prompt to the LLM and get the response
    response = send_prompt_to_llm(prompt)

    # Store the prompt and response in the dictionary
    responses_dict[prompt] = response
```

Listing 4.4: Sending Prompts to LLM and Storing Responses in Dictionary

5 Discussion

Iterating values like "lastNameïs straightforward, but creating accurate or realistic values for more complex fields such as names, addresses, and IBANs requires additional tools. For names, we could rely on a long enough list sourced from any online platform, or alternatively, generate them using \ac{LLM}s, which can offer a degree of variety and realism. However, IBANs pose a different challenge, as they require specific logic due to banking standards and validation processes. While OpenAI's GPT-4 managed to produce a valid IBAN for testing, the question remains whether it can consistently generate unique and valid IBANs at scale.

This brings up an interesting consideration: do all fields need to be unique in every instance? For most fields, the system is flexible enough to accept duplicates. However, specific cases like \ac{MaLo} and \ac{MeLo} demand unique IDs to avoid conflicts in the billing and metering processes. These fields, though, follow a straightforward format, consisting of 11-digit numbers, which can be iterated easily. The system doesn't need sophisticated logic for such cases, just a reliable method of ensuring each ID is unique.

Now, if we consider fields such as names, addresses, and bank details, the requirement for uniqueness is not always necessary. The system can handle non-unique values in these cases without much issue. However, if we were tasked with ensuring uniqueness across these fields—particularly for a high-volume dataset—then it might be worth investigating whether open-source \ac{LLM}s or other AI-based tools can generate such data accurately at scale. That being said, my personal opinion is that we don't always need a hammer like Mjolnir to drive a nail into the wall. The problem might not warrant such an advanced solution, especially when simpler methods, such as using pre-generated data sets or iterating values, might suffice.

In summary, while the project demonstrates the utility of iterating values for many fields, the need for uniqueness depends on the context. For some fields, more advanced

techniques could be explored, but they may not be necessary for every scenario. Simpler solutions might be more efficient, depending on the specific requirements of the testing environment.

6 Conclusion

This project successfully demonstrated a Python-based solution to define and structure TDCs for the TAP SAP testing environment, tackling the complex task of generating test data constructs. By categorizing required fields and utilizing simple iteration techniques for straightforward values like last names, the solution provides a scalable, yet efficient approach. For more complex fields, such as names, addresses, and IBANs, the project explored advanced methods like online generators and LLMs, which offer flexibility in data generation.

While LLMs like GPT-4 showed potential in generating realistic data samples, their true effectiveness at scale remains an open question. Initial tests suggest that LLMs can handle small batches of data effectively, but further research is required to determine whether these models can sustain performance and accuracy when generating large datasets. This is particularly important when considering the necessity for uniqueness in fields such as IBANs, where specific validation rules must be followed.

Ultimately, the project underscores the importance of balancing simplicity and complexity in test data generation. Iterative methods prove reliable for most fields, while advanced techniques may be reserved for scenarios where accuracy and uniqueness are critical. However, as with any tool, knowing when and where to apply more sophisticated methods, such as LLMs, is key. Although these advanced techniques offer promise, their practicality and efficiency must be evaluated in greater detail, especially in high-volume testing environments. Overall, this solution provides a robust foundation for automating and managing test data within SAP's BilP, with room for future expansion as technology advances.

List of Figures

4.1 Creating a BP 12
4.2 Extracting BP Master Data 15
4.3 Extracting Address Data 18

List of Tables

3.1 Tcodes and Sample Data 6

4.1 Field Names and Values Table 12
4.2 BP Fields and Data Categories 16

Listings

4.1 Mapped Python Dictionary for Business Partner 16
4.2 Last Name Iterator Function 17
4.3 Curl Request for Random German Profiles 18
4.4 Sending Prompts to LLM and Storing Responses in Dictionary 19

Literaturverzeichnis

AG, T. (n.d.), Thüga und accenture entwickeln abrechnungsplattform für die energiewirtschaft. Accessed: 2024-09-09. Available: https://www.thuega.de/stadtwerke-der-zukunft/thuga-und-accenture-entwickeln-abrechnungsplattform-fur-die-energiewi

Anonymous (2024), Are llms naturally good at synthetic tabular data generation? Preprint.
URL: *https://arxiv.org/abs/2406.14541v2*

apilayer (n.d.), 'OpenIBAN', https://openiban.com/. Accessed: 2024-09-09.

de Lima Junior rflj@cesar.org.br Luiz Fernando Paes de Barros Presta lfpbp@cesar.school Lucca Santos Borborema lsb@cesar.school Vanderson Nogueira da Silva vns@cesar.org.br Marcio Leal de Melo Dahia mlmd@cesar.org.br Anderson Carlos Sousa e Santos acss@cesar.org.br, R. F. (2023), 'A case study on test case construction with large language models: Unveiling practical insights and challenges', *arxiv* **2**(2), 9.

Fakexy.com (2024), Fake address generator germany. Website accessed September 9, 2024.
URL: *https://www.fakexy.com/fake-address-generator-de*

Molina-Castillo, F.-J., Rodríguez, R., López-Nicolas, C. and Bouwman, H. (2022), 'The role of erp in business model innovation: Impetus or impediment', *Digital Business* **2**(2), 100024.
URL: *https://www.sciencedirect.com/science/article/pii/S2666954422000047*

OpenAI (2024), Research. Online Resource.
URL: *https://openai.com/research/*

Panaya (2024), 'Sap transaction codes: The ultimate testing checklist'. Accessed: 2024-09-06.
URL: *https://www.panaya.com/blog/sap/sap-transaction-codes-ultimate-testing-checklist/*

RapidAPI (2024), Random german profiles and names generator. Website accessed September 9, 2024.
URL: *https://rapidapi.com/dawn2k/api/random-german-profiles-and-names-generator/playground/57511e56e4b02d863afe6344*

Seren, D. (2024), 'Billing in sap client 300 + client 400', Internal TAP/Accenture Documentation. Unpublished internal document.

TechTarget (2023), 'What is sap s/4hana?'. Accessed: 6 September 2024.

List of Abbreviations

BilP	Billable Process
ERP	Enterprise Resource Planning
Tcode	Transaction Code
TAP	Thüga-Abrechnungsplattform
TDC	Test Data Construct
MaLo	Market Location
MeLo	Meassurement Location
LLM	Large Language Model
curl	Client URL
JSON	JavaScript Object Notation